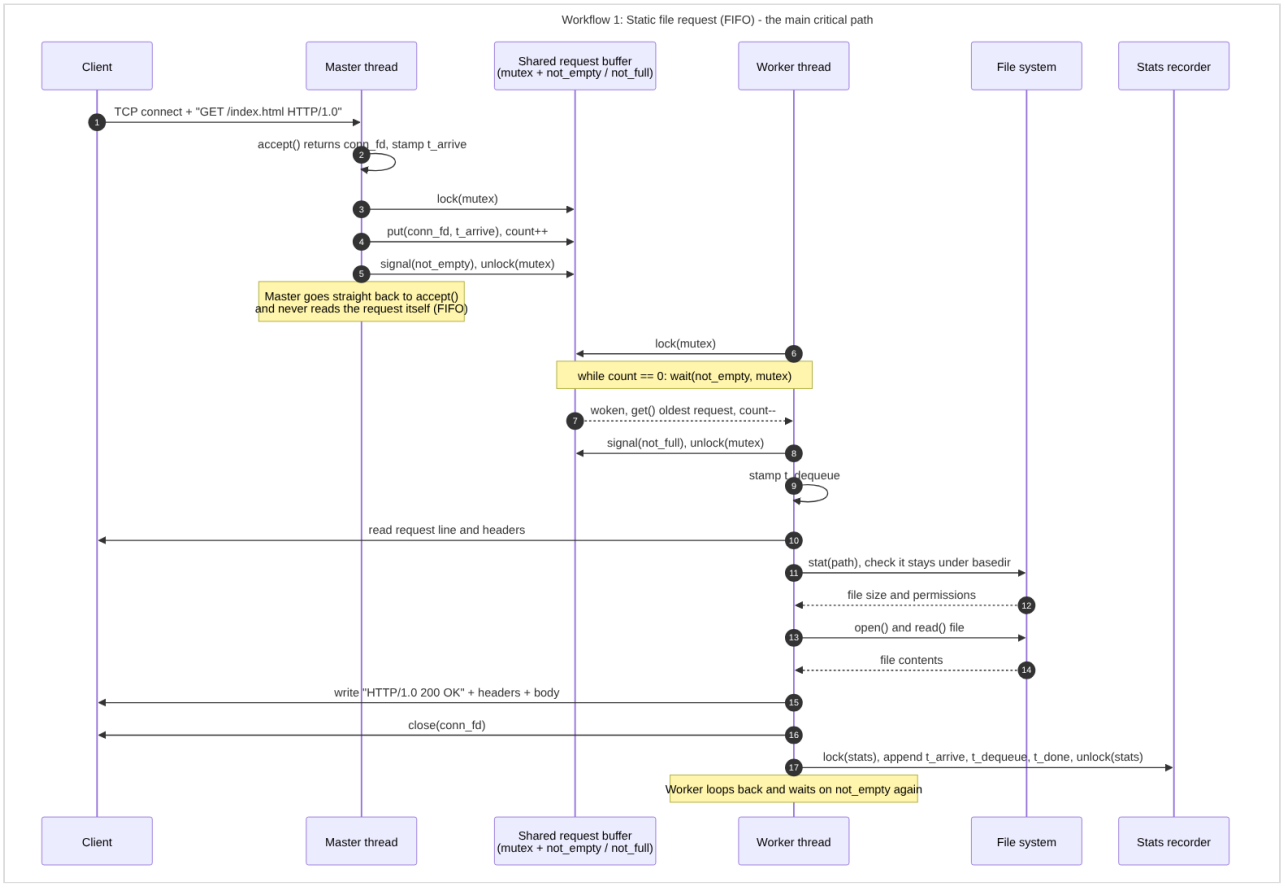
System Architecture: UML Sequence Diagrams

Our server follows the OSTEP Project G.5 design: a master thread accepts connections and puts them into a bounded buffer, and a pool of worker threads takes requests out and serves them (producer/consumer, one mutex and two condition variables). The three diagrams below show three different execution paths through this design.

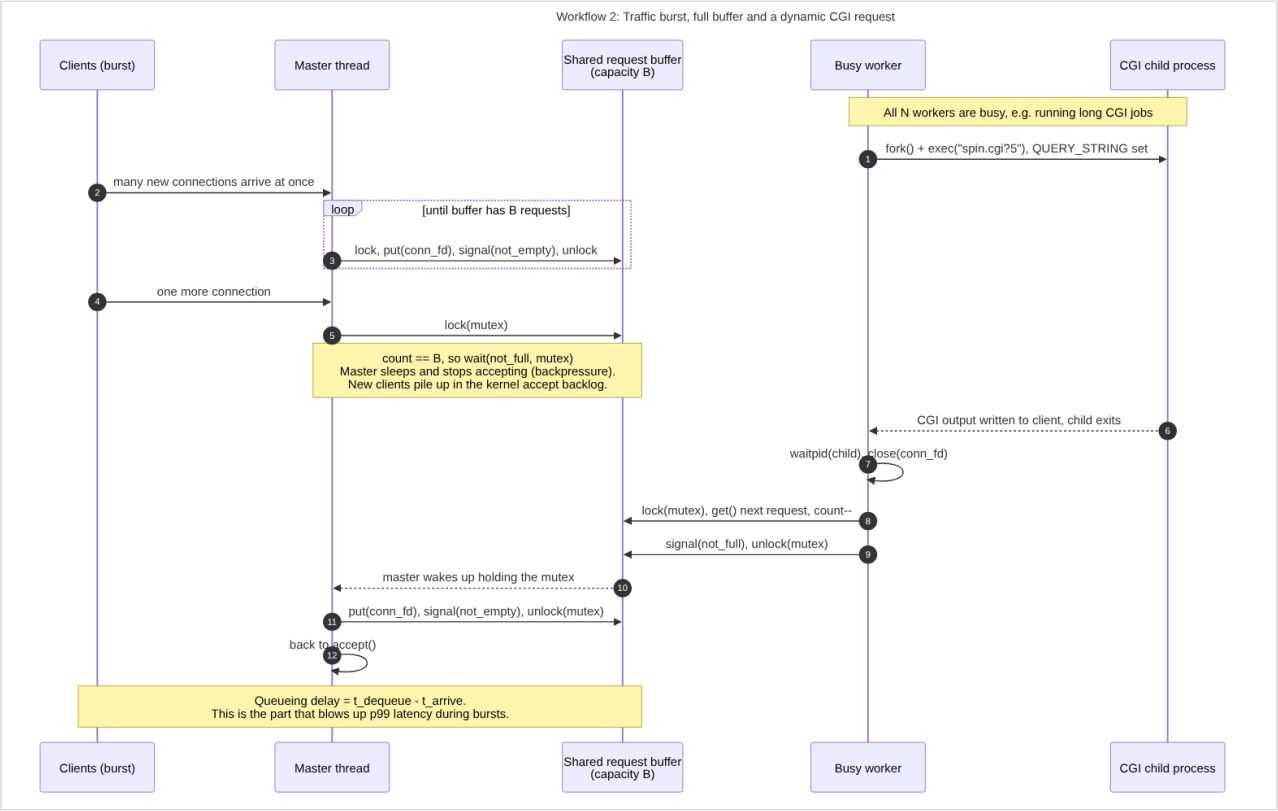
Workflow 1: Static file request (FIFO)

This is the most frequently executed path in the server and the main critical path. The master thread only accepts the connection and enqueues it; all request parsing, file I/O and response writing happen on a worker. The two hot spots are the shared buffer's mutex, which every request takes twice (enqueue and dequeue), and the stats lock. Under SFF the master also reads the request and calls stat() before enqueueing, so it can order the buffer by file size.



Workflow 2: Traffic burst, full buffer and a dynamic CGI request

When every worker is busy (here, running long CGI programs through fork/exec/waitpid) and a burst fills the buffer, the master blocks on the not_full condition variable and stops accepting connections. New clients then wait in the kernel's accept backlog. This path is where queueing delay, and therefore p99 latency, grows during bursts. It is the problem our research extension targets.



Workflow 3 (research extension): Adaptive thread pool resizing

A monitor thread samples the queue length and the number of idle workers on a fixed interval. If the queue stays above a high watermark, it creates new workers quickly, up to a maximum. If workers stay idle and the queue is empty for a longer window, it sets a retire target and broadcasts on not_empty, so an idle worker exits safely between requests and never in the middle of one. Growing and shrinking use different thresholds (hysteresis) so the pool does not flap.

